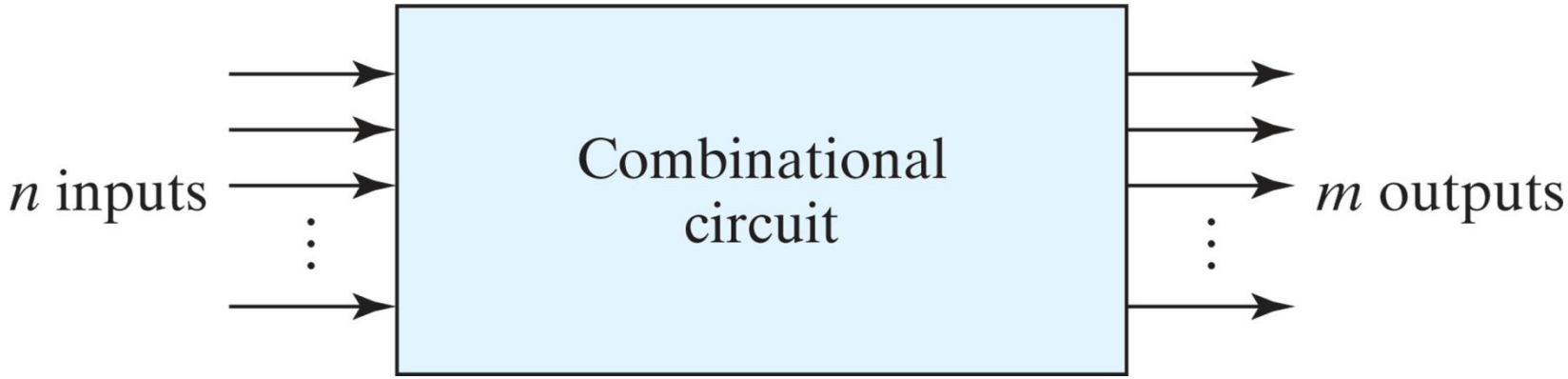
Digital Electronic Circuits (EC2L004)

Lecture 8: Combinational Logic – Part I

Introduction

- A combinational circuit performs an operation that can be specified logically by a set of Boolean functions.
- In contrast, sequential circuits employ storage elements in addition to logic gates. Their outputs are a function of the inputs and the state of the storage elements.
- A combinational circuit consists of an interconnection of logic gates.



Block diagram of a combinational circuit

Combinational Circuits

- A combinational circuit can be described by *m* Boolean functions, one for each output variables.
  - Each output function is expressed in terms of the *n* input variables.

- Most important standard combinational circuits are:

  - Adders
  - Subtractors
  - Comparators
  - Decoders
  - Encoders
  - Multiplexers

Available in IC's as MSI and used as *standard cells* in complex VLSI (ASIC)

Analysis Procedure

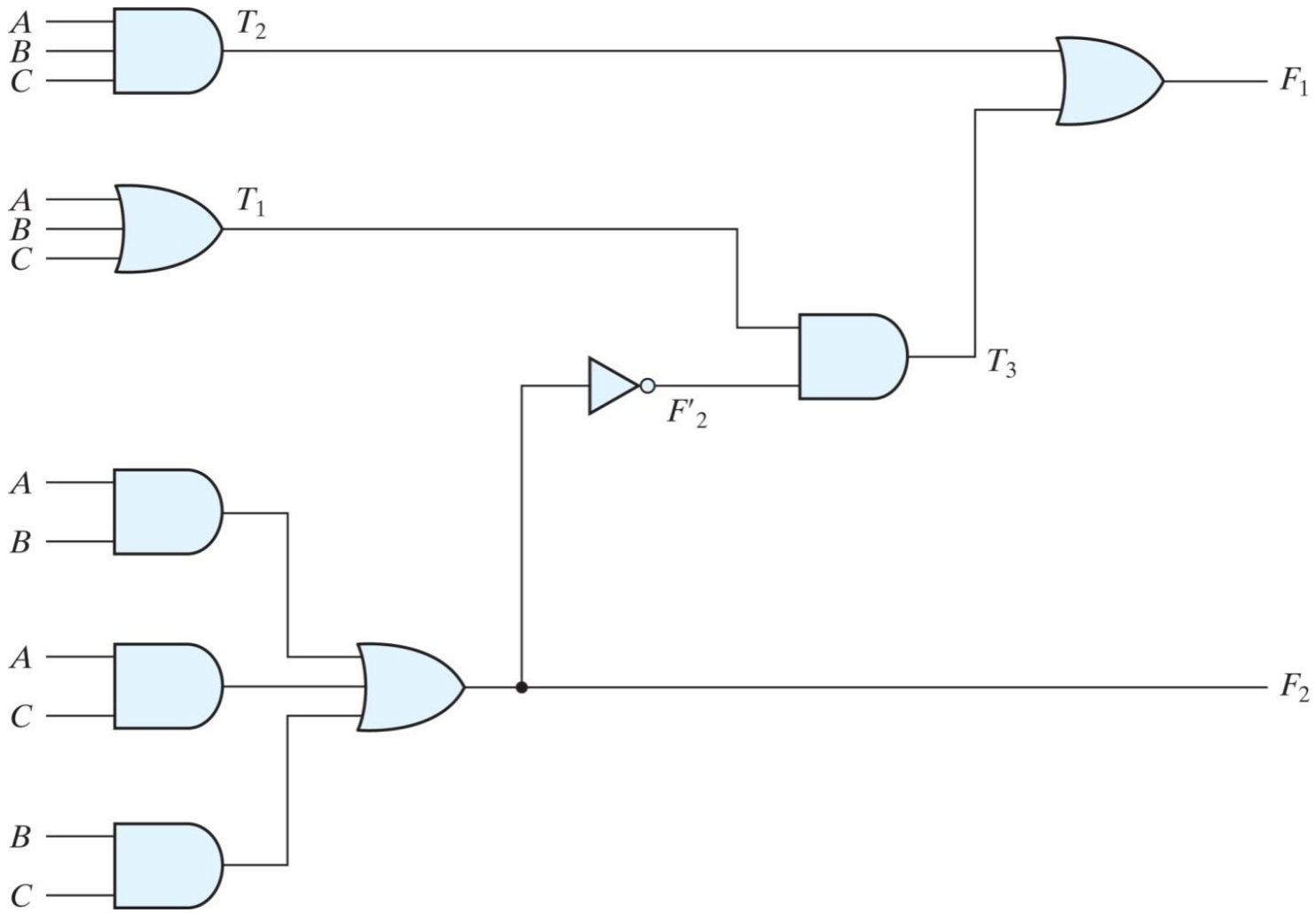
- The analysis of a combinational circuit requires that we determine the function that the circuit implements.

- The first step in the analysis is to make the sure that the given circuit is combinational and not sequential.

  - The diagram of a combinational circuit has logic gates with no feedback paths or memory elements.

- This task starts with a given logic diagram and culminates with a set of Boolean functions, a truth table, or, possibly, an explanation of the circuit operation.

Logic Diagram for Analysis Example



Example (Contd.)

$$F_1 = T_3 + T_2 = F_2'T_1 + ABC$$

$$= (AB + AC + BC)'(A + B + C) + ABC$$

$$= (A'+B')(A'+C')(B'+C')(A + B + C) + ABC$$

$$= (A'+B'C')(AB'+AC'+BC'+B'C) + ABC$$

$$= A'BC' + A'B'C + AB'C' + ABC$$

$$F_2 = AB + AC + BC$$

Example (Contd.)

Truth Table for the Logic Diagram

A	B	C	F_2	F'_2	T_1	T_2	T_3	F_1
0	0	0	0	1	0	0	0	0
0	0	1	0	1	1	0	1	1
0	1	0	0	1	1	0	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	1	1	0	1	1
1	0	1	1	0	1	0	0	0
1	1	0	1	0	1	0	0	0
1	1	1	1	0	1	1	0	1

- From the truth table can you tell the function of the circuit ?

Verilog Code

```
module analysis (A,B,C,F1,F2);
    input A,B,C;
    output F1,F2;
    wire T1,T2,T3,F2not,E1,E2,E3;
    or g1 (T1,A,B,C);
    and g2 (T2,A,B,C);
    and g3 (E1,A,B);
    and g4 (E2,A,C);
    and g5 (E3,B,C);
    or g6 (F2,E1,E2,E3);
    not g7 (F2not,F2);
    and g8 (T3,T1,F2not);
    or g9 (F1,T2,T3);
endmodule
```

Verilog Test bench

```
//Stimulus to analyze the circuit
module test_circuit;
reg [2:0]D;
wire F1,F2;
analysis circuit(D[2],D[1],D[0],F1,F2);
// D[2]=A, D[1]=B, D[0]=C
initial
begin
D = 3'b000;
repeat(7)
#10 D = D + 1'b1;
end
initial
$monitor ("ABC = %b F1 = %b F2 =%b ",D, F1, F2);
//Display truth table
endmodule
```

Simulation log: ABC = 000 F1 = 0 F2 =0
ABC = 001 F1 = 1 F2 =0 ABC = 010 F1 = 1 F2 =0
ABC = 011 F1 = 0 F2 =1 ABC = 100 F1 = 1 F2 =0
ABC = 101 F1 = 0 F2 =1 ABC = 110 F1 = 0 F2 =1
ABC = 111 F1 = 1 F2 =1

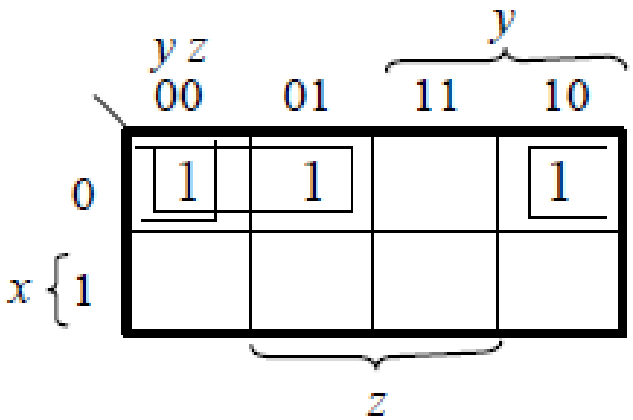
Design Procedure

- From the specifications of the circuit, determine the number of inputs and outputs
- Derive the truth table that defines the relationship between the input and the output.
- Obtain the simplified Boolean function using *x*-variable K-Map.
- Draw the logic diagram and verify the correctness of the design (manually or by simulation).

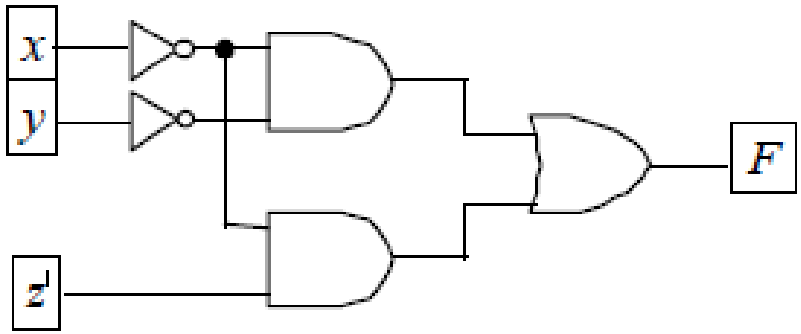
Design Example 1:

- Design a combinational circuit with three inputs and one output. The output is a 1 when the binary value is less than three. The output is 0 otherwise.

x	y	z	F
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0



$$F = x'y' + x'z'$$



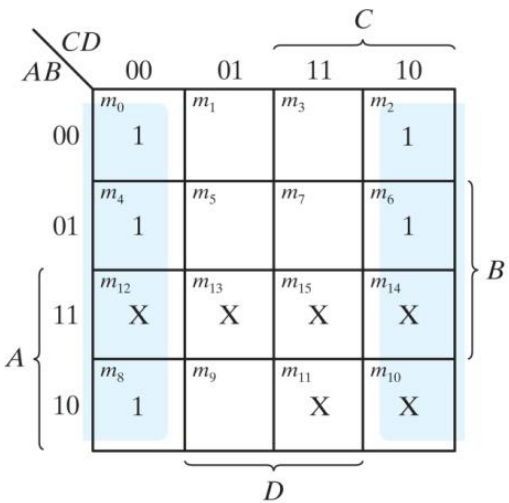
Design Example 2:

- Design a code converter that converts a BCD number to excess-3 code.

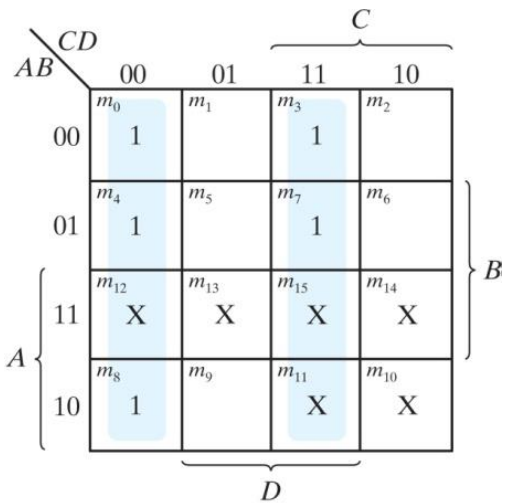
Truth Table for Code Conversion Example

Input BCD				Output Excess-3 Code			
A	*B*	*C*	*D*	*w*	*x*	*y*	*z*
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

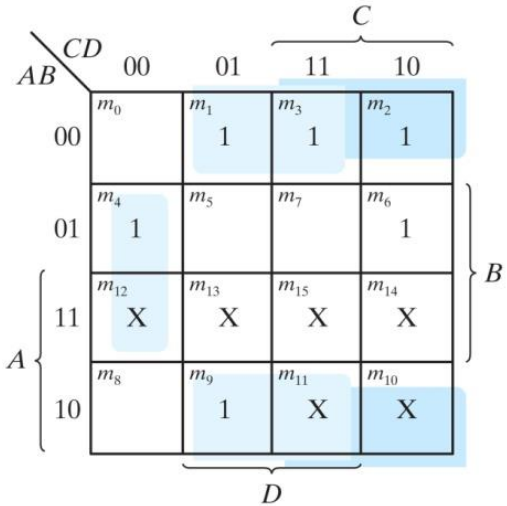
Maps for BCD-to-excess-3 code converter



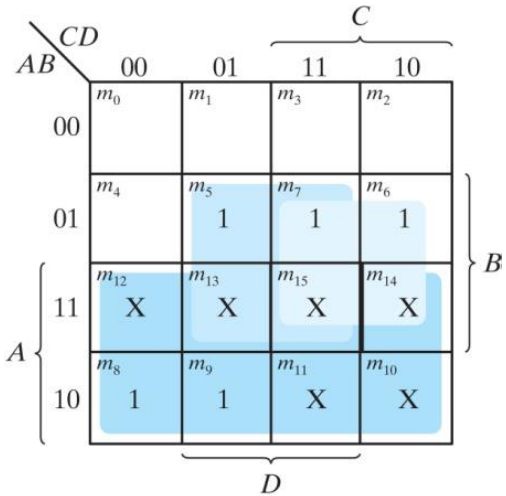
$z = D'$



$y = CD + C'D'$

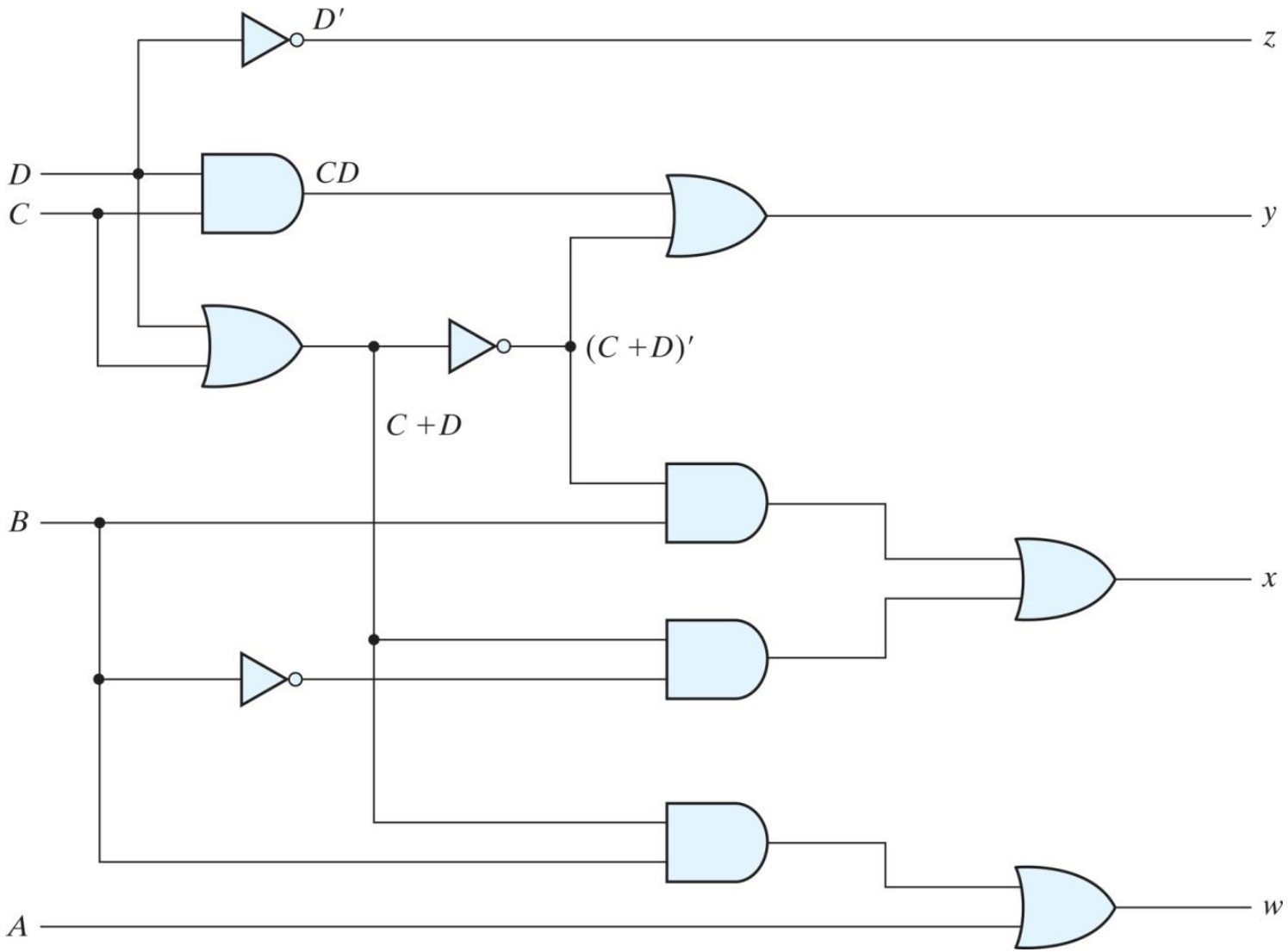


$x = B'C + B'D + BC'D'$



$w = A + BC + BD$

Logic Diagram: BCD to Excess-3 code converter



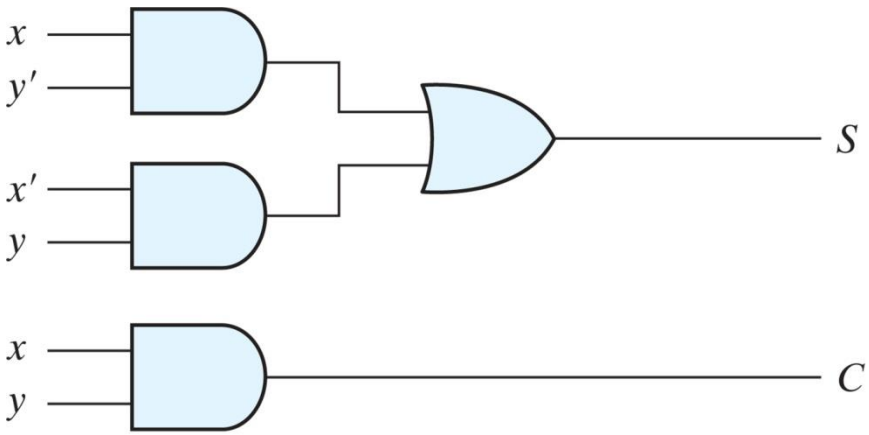
Binary Adder

- Half Adder
  - A combinational circuit that performs the addition of two bits is called a "*half adder*".

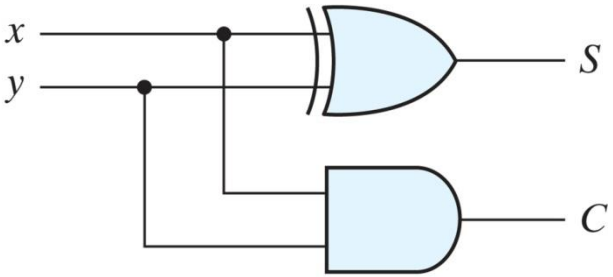
Block diagram of half adder

Half Adder

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



(a) $S = xy' + x'y$
$C = xy$

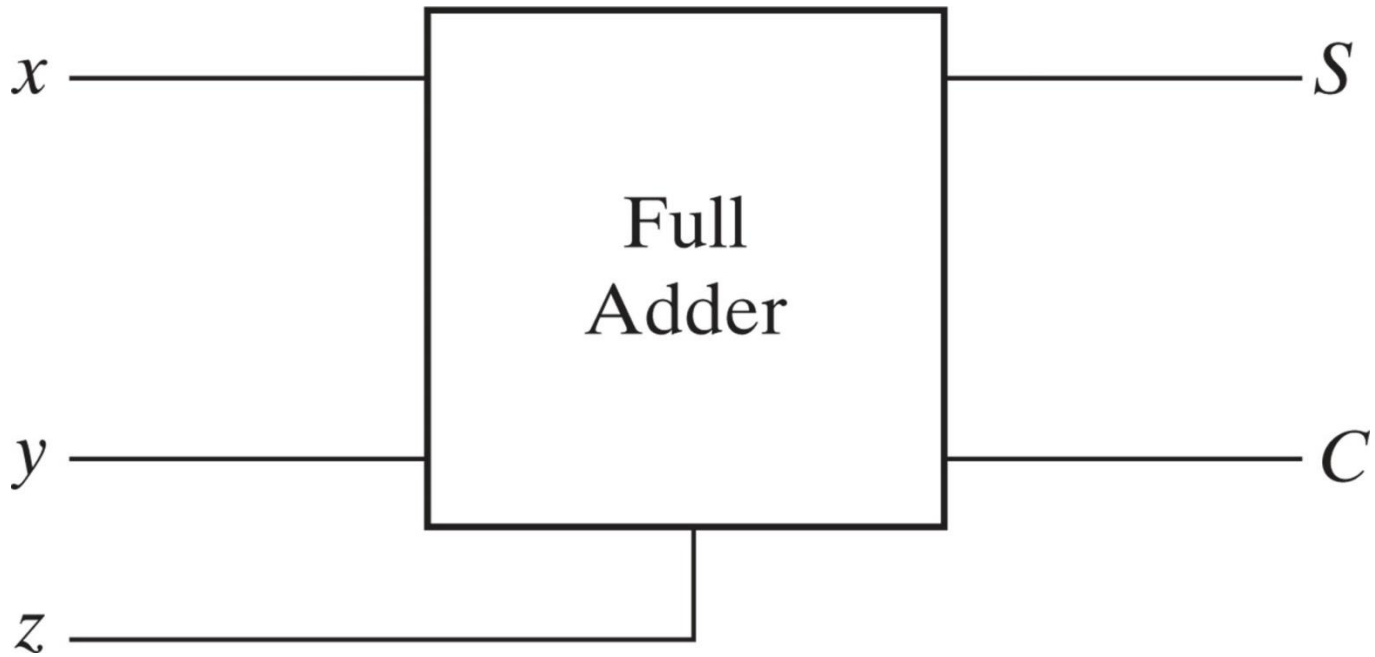


(b) $S = x \oplus y$
$C = xy$

Implementation of half adder

Full Adder

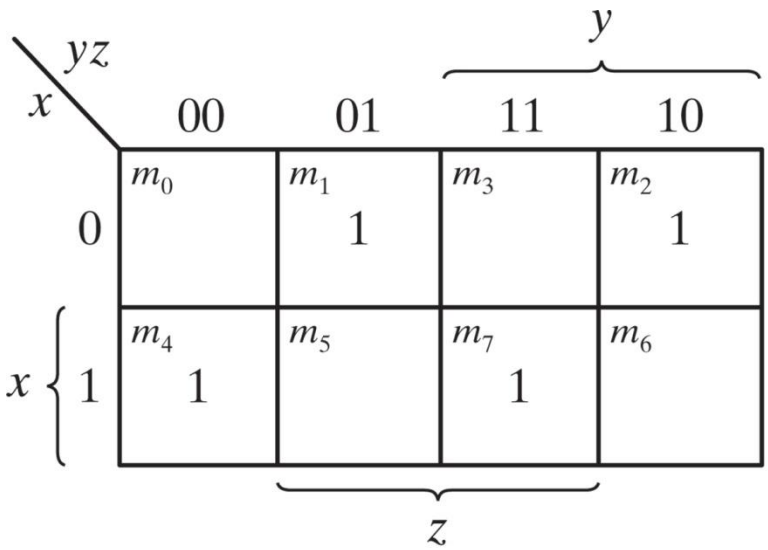
- A combinational circuit that performs the addition of three bits (two significant bits and a previous carry) is a full adder.



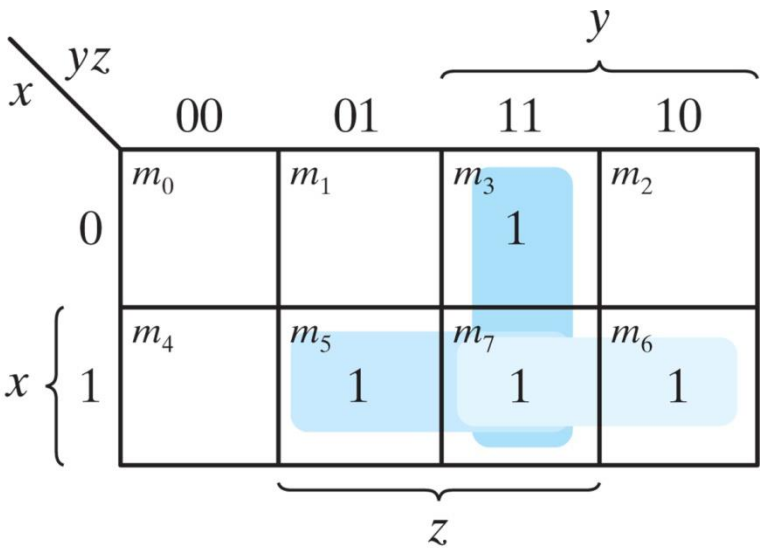
Block diagram of full adder

Full Adder (Contd.)

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

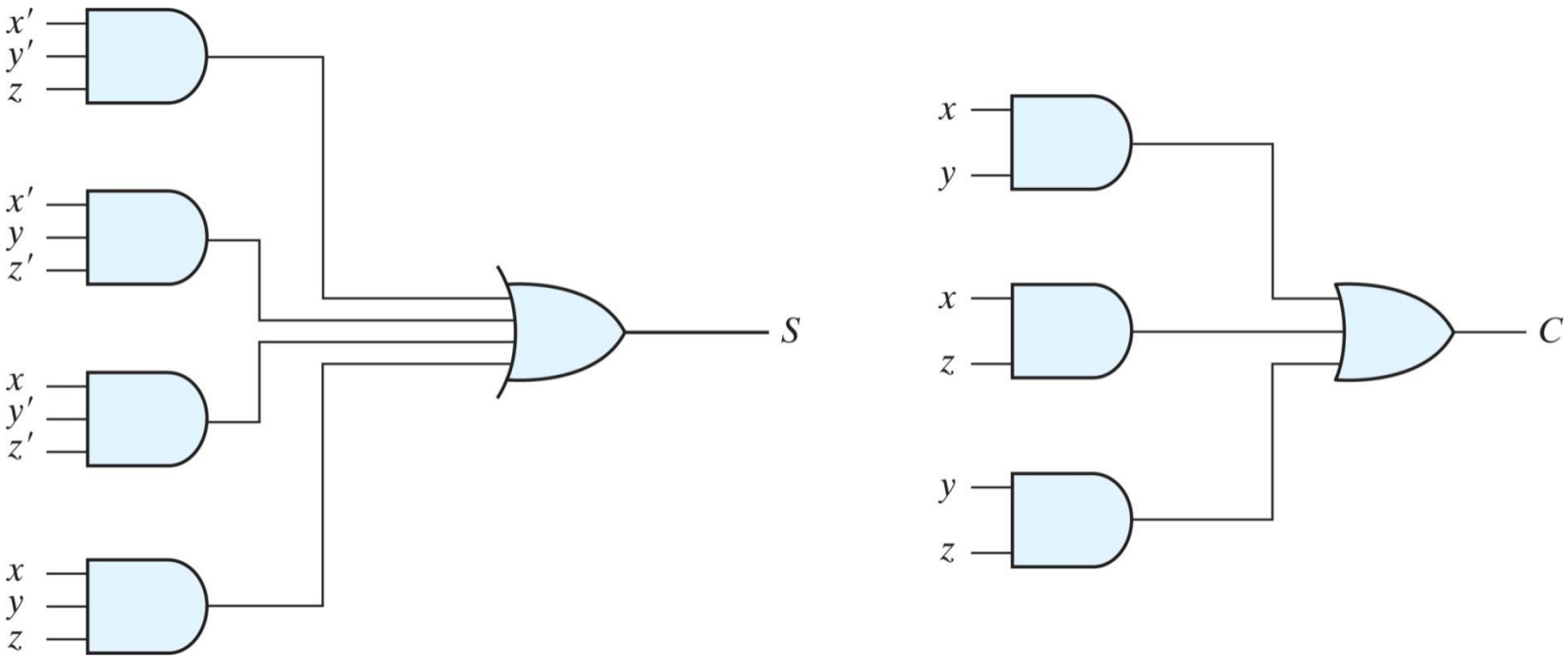


(a) $S = x'y'z + x'yz' + xy'z' + xyz$



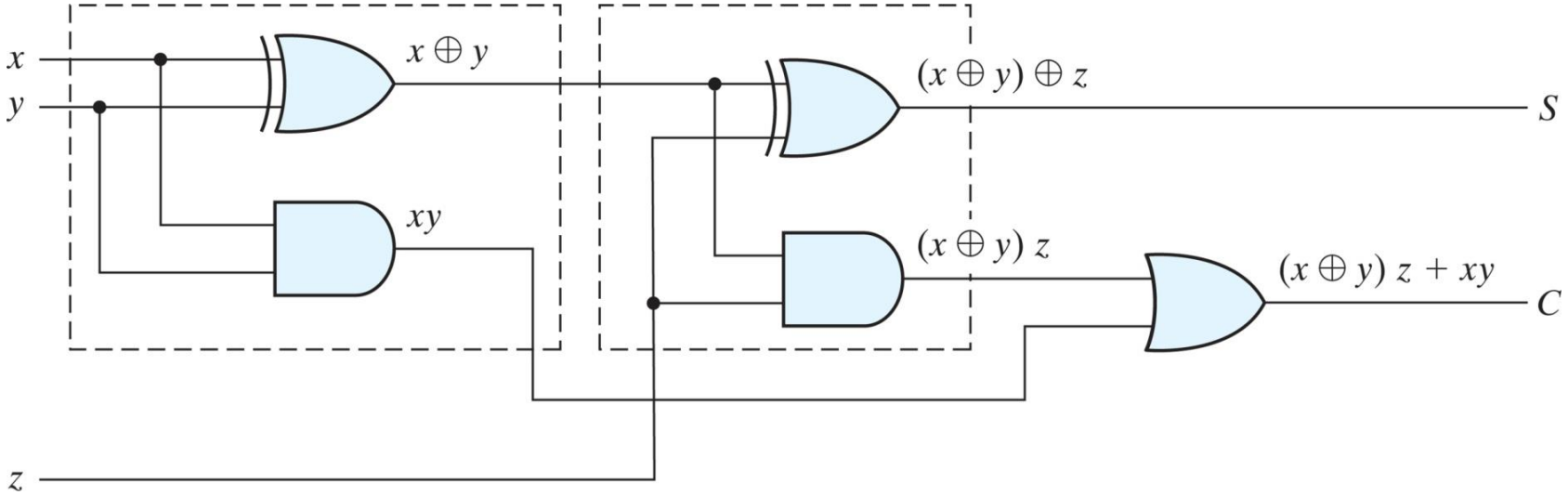
(b) $C = xy + xz + yz$

Full Adder (Contd.)



Implementation of full adder in sum-of-products form

Full Adder (Contd.)



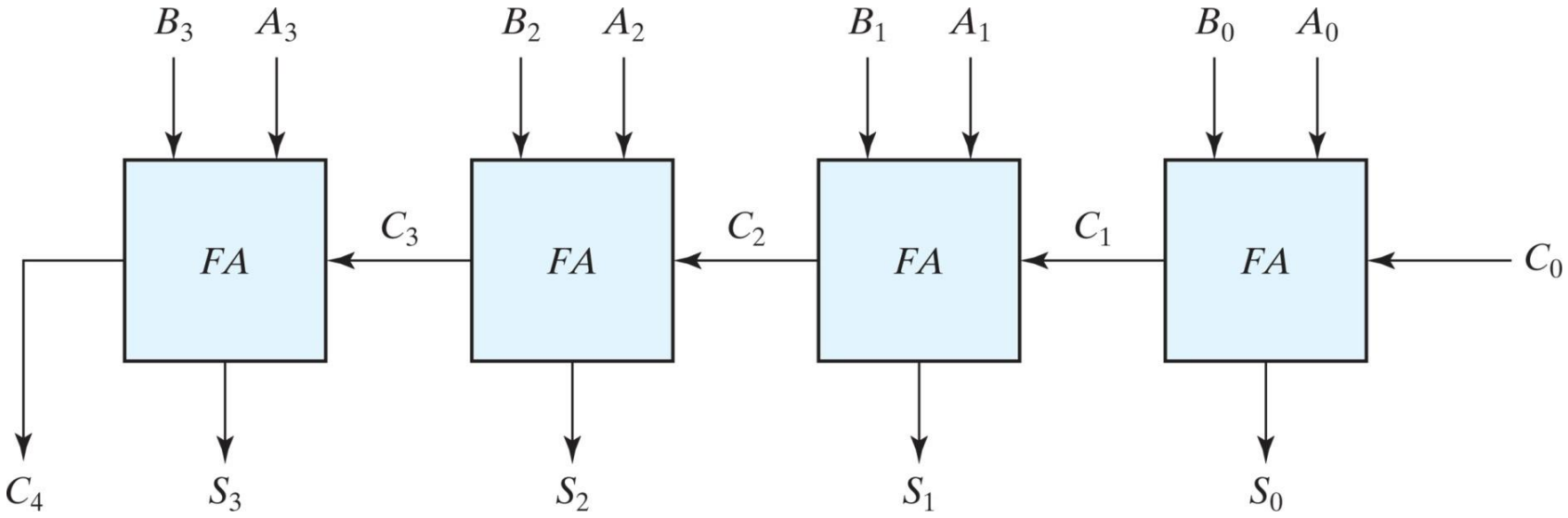
Implementation of full adder with two half adders and an OR gate

Binary Adder

- A binary adder is a digital circuit that produces the arithmetic sum of two binary numbers.
  - It can be constructed with full adders connected in cascade, with the output carry from each full adder connected to the input carry of the next full adder in the chain.
  - Addition of *n*-bit numbers requires a chain of *n* full adders or a chain of half adder and *n-1* full adders.

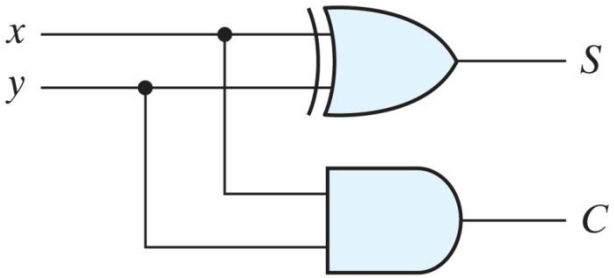
Subscript *i*	3	2	1	0	
Input carry	0	1	1	0	C_i
Augend	1	0	1	1	A_i
Addend	0	0	1	1	B_i
Sum	1	1	1	0	S_i
Output carry	0	0	1	1	C_{i+1}

4-bit Binary Ripple Carry Adder



Verilog Codes: Half Adder, Full Adder

```
//Gate-level hierarchical description of 4-bit adder
//Description of half adder
module halfadder (S,C,x,y);
input x,y;
output S,C;
//Instantiate primitive gates
xor (S,x,y);
and (C,x,y);
endmodule
```



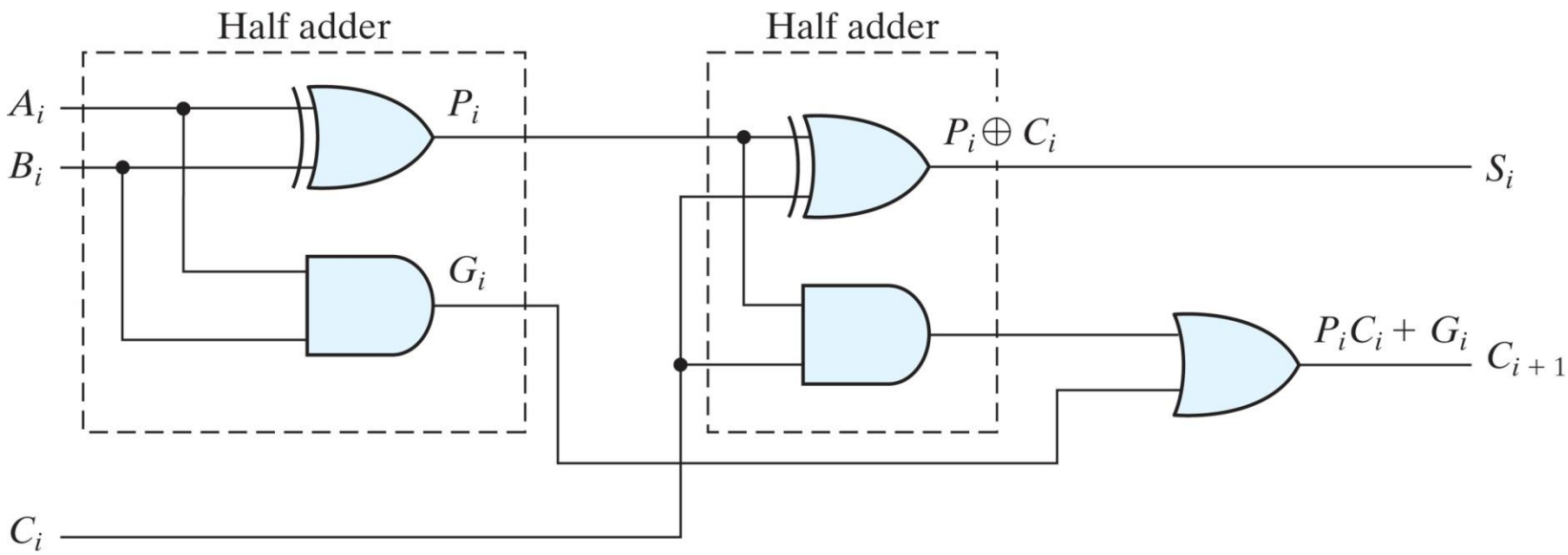

```
//Description of full adder
module fulladder (S,C,x,y,z);
input x,y,z;
output S,C;
wire S1,D1,D2; //Outputs of first XOR and two AND gates
//Instantiate the halfadder
halfadder HA1 (S1,D1,x,y),
HA2 (S,D2,S1,z);
or g1(C,D2,D1);
endmodule
```

Verilog Codes: 4-bit ripple carry adder

```
//Description of 4-bit adder
module ripple_carry_4bit_adder (S,C4,A,B,C0);
input [3:0] A,B;
input C0;
output [3:0] S;
output C4;
wire C1,C2,C3; //Intermediate carries
//Instantiate the fulladder
fulladder FA0 (S[0],C1,A[0],B[0],C0),
FA1 (S[1],C2,A[1],B[1],C1),
FA2 (S[2],C3,A[2],B[2],C2),
FA3 (S[3],C4,A[3],B[3],C3);
endmodule
```

Carry Propagation

- The carry propagation time is a limiting factor on the speed with which two numbers are added.
- The most widely technique for reducing the carry propagation time in a parallel adder uses the principle of *carry lookahead*.



Carry Lookahead Adder

- Algorithm

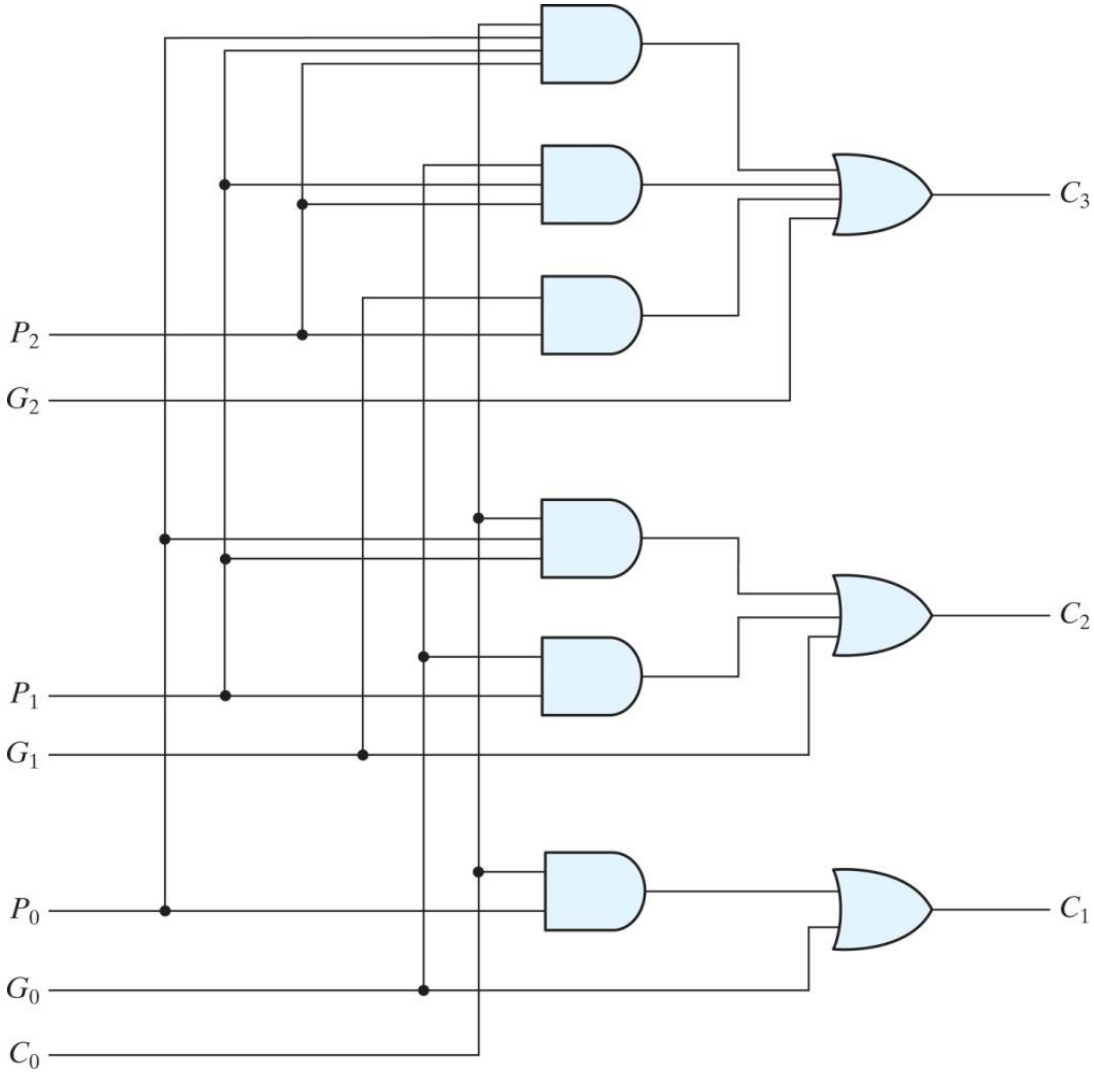
$$\textit{Carry Propagate} \quad P_i = A_i \oplus B_i \qquad\qquad S_i = P_i \oplus C_i$$

$$\textit{Carry Generate} \quad G_i = A_i B_i \qquad\qquad C_{i+1} = G_i + P_i C_i$$

- G_i is called a carry generate, and it produces a carry of 1 when both A_i and B_i are 1, regardless of the input carry C_i.

- *Pi* is called a carry propagate, because it determines whether a carry into stage *"i"* will propagate into stage *"i+1".*

Carry Lookahead Adder (Contd.)



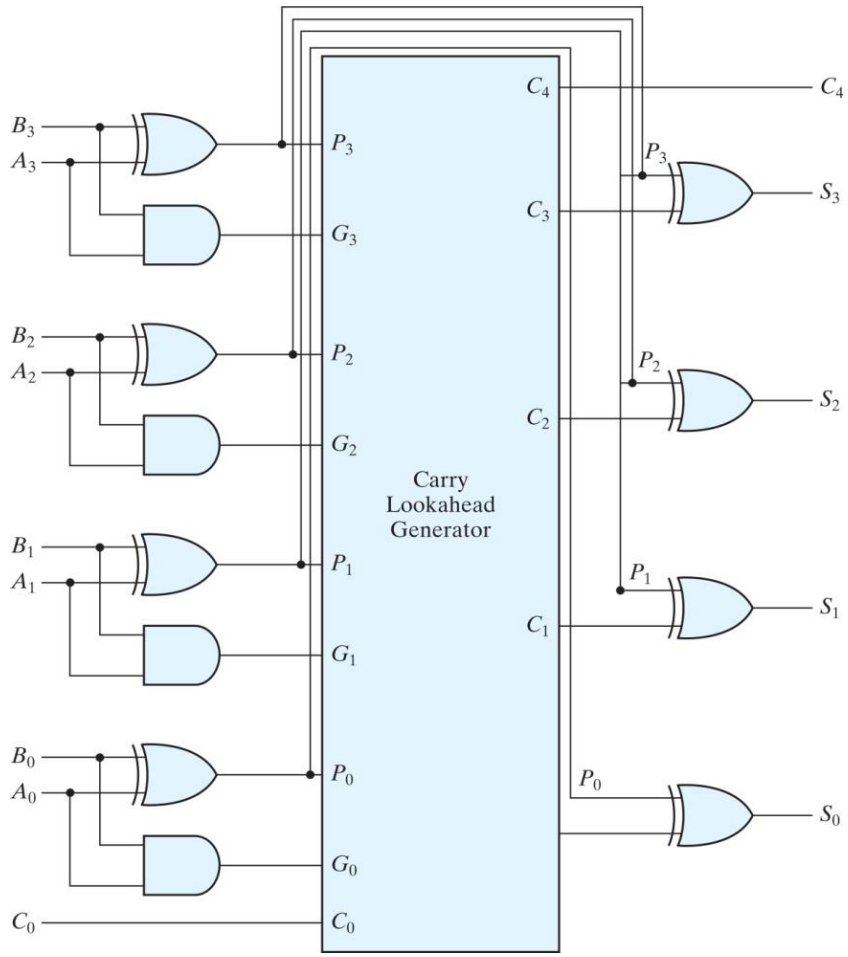
C_0 = input carry

$C_1 = G_0 + P_0C_0$

$C_2 = G_1 + P_1C_1 = G_1 + P_1(G_0 + P_0C_0) = G_1 + P_1G_0 + P_1P_0C_0$

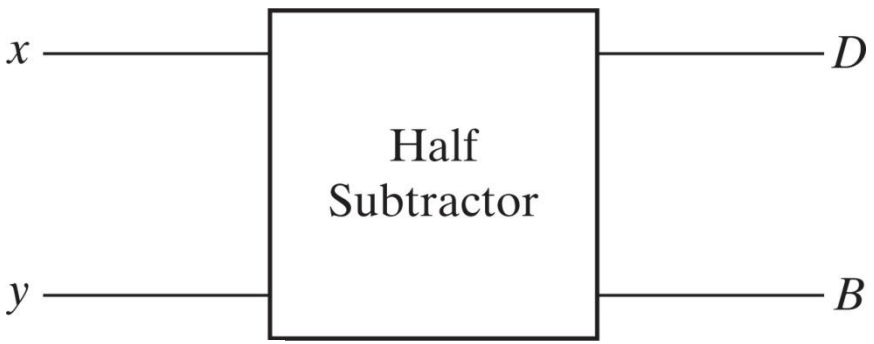
$C_3 = G_2 + P_2C_2 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$

Carry Lookahead Adder (Contd.)



Four-bit adder with carry lookahead

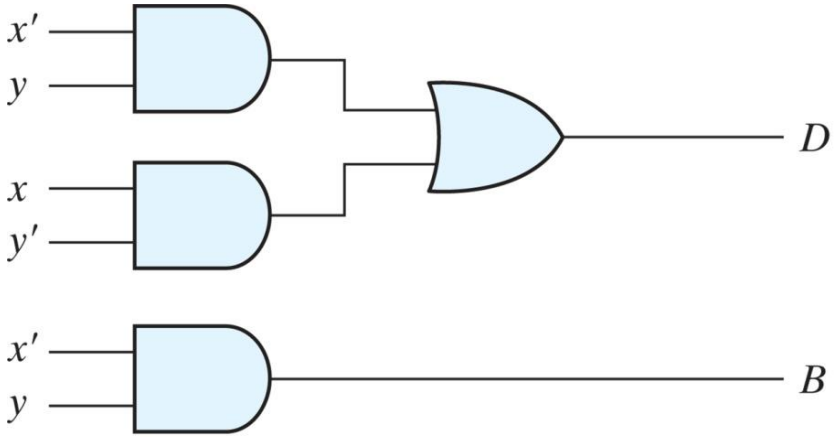
Half Subtractor



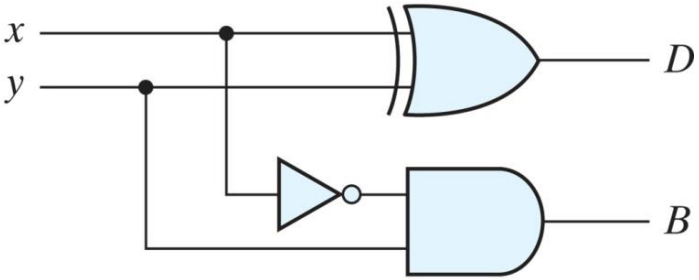
Half Subtractor

x	y	D	B
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Half Subtractor (Contd.)



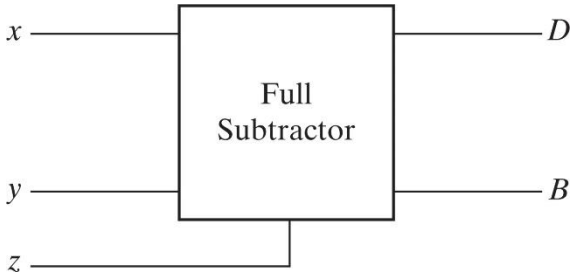
(a) $D = x'y + xy'$, $B = x'y$



(b) $D = x \oplus y$
$B = x'y$

Full Subtractor

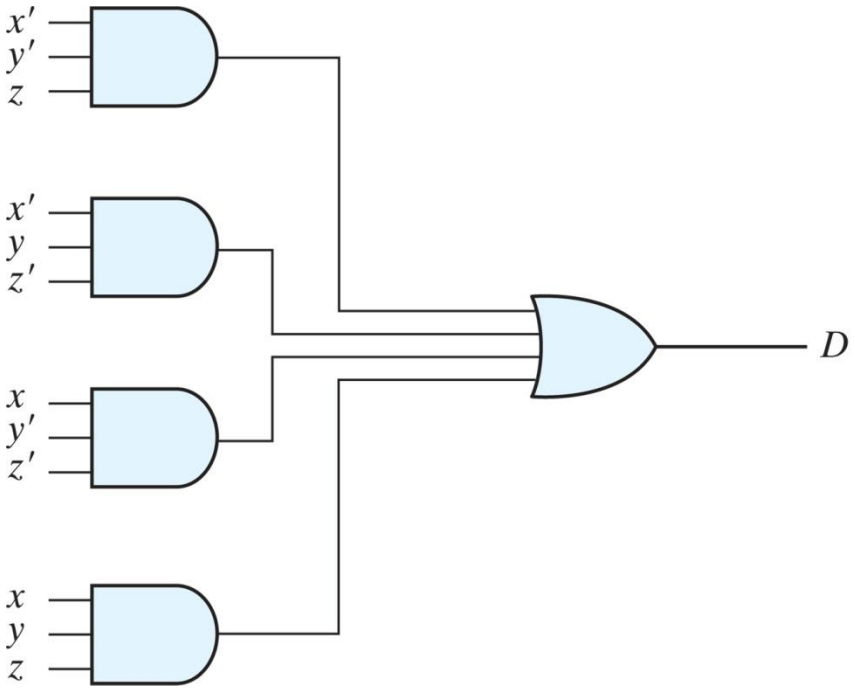
- A full subtractor has three inputs and two outputs. *x*, *y*, and *z* are the inputs to be subtracted in which *z* represents borrow from the next stage.

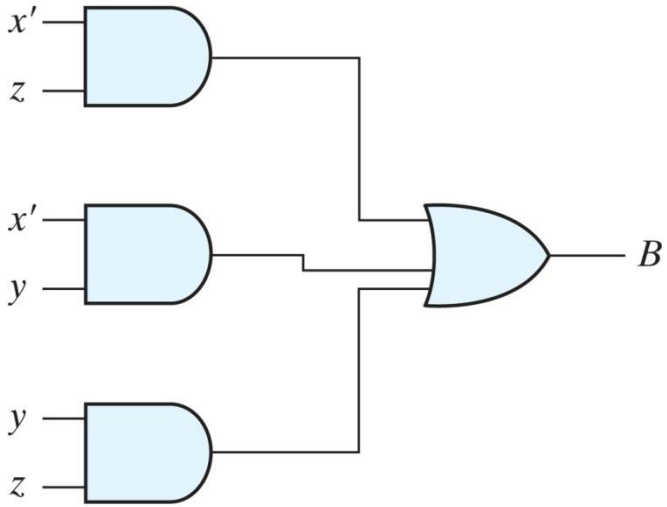


Full Subtractor

x	y	z	D	B
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Full Subtractor (Contd.)





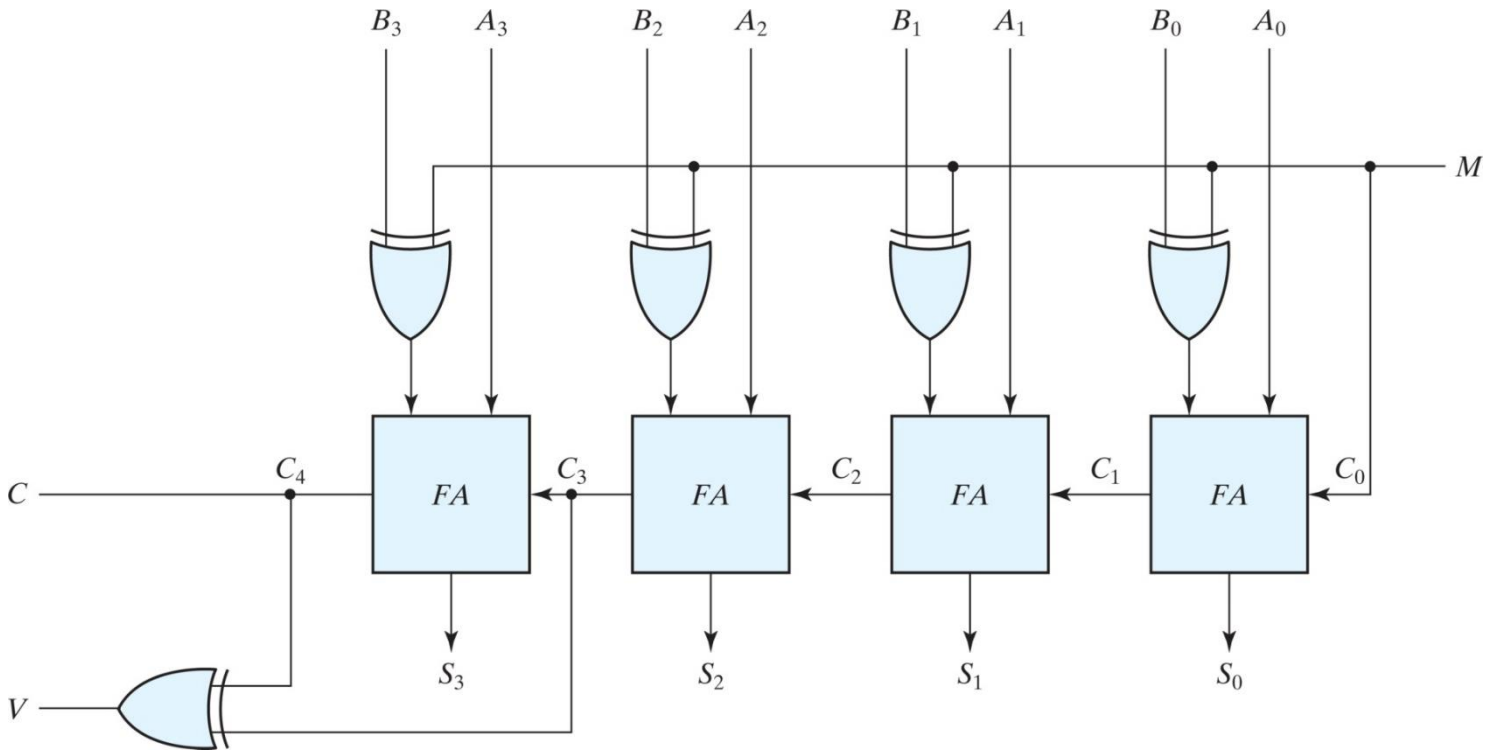
Binary Subtractor

- The subtraction of unsigned binary numbers can be done by complements.

Review complements:

- 1's complement of $N = (2^n-1) - N$ (N is a binary #)
- 1's complement can be formed by changing 1's to 0's and 0's to 1's
- 2's complement of a number is obtained by leaving all least significant 0's and the first 1 unchanged, and replacing 1's with 0's and 0's with 1 in all higher significant digits.
    - Taking the 1's complement and adding 1 to the least significant bit in the number.

Binary Adder/Subtractor



Line M controls the the operation (addition or subtraction)

$M = 0$ → Adder: A plus B, $C_0 = 0$

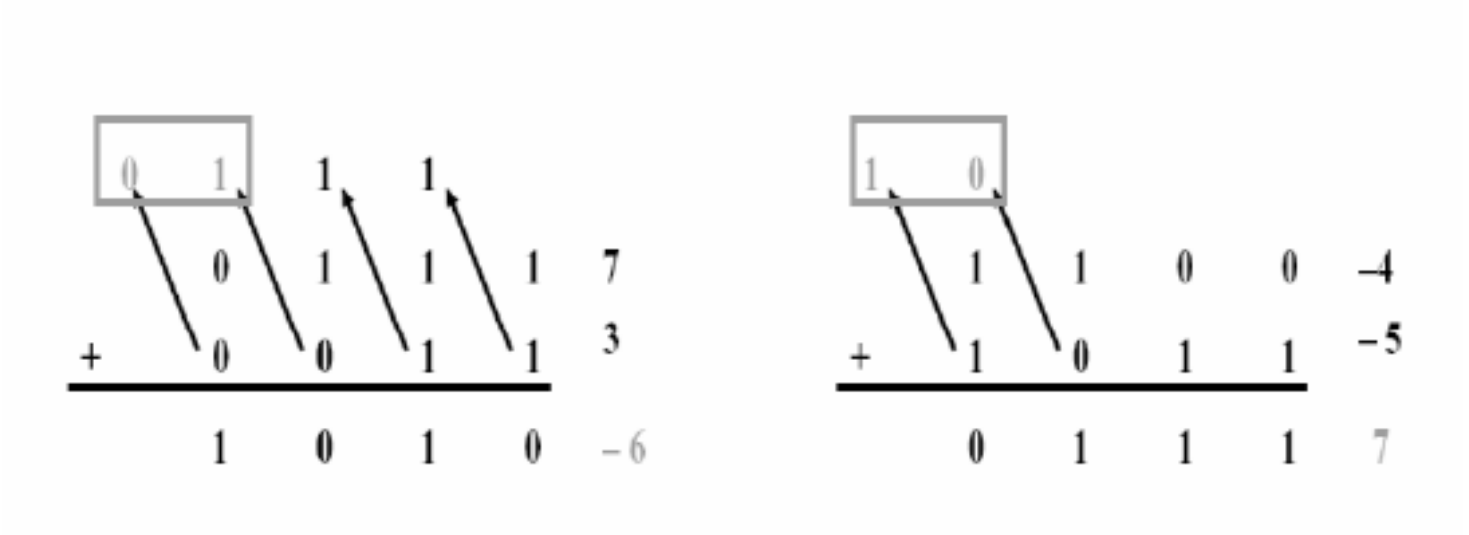
$$B \oplus 0 = (B \cdot 1) + (B' \cdot 0) = B$$

$M = 1$ → Adder: A plus 2's complement and a $C_0 = 1$ → $A - B$

$M = 1$: $B \oplus 1 = (B \cdot 0) + (B' \cdot 1) = B'$

Binary Adder/Subtractor (Contd.)

- Overflow occurs when two numbers of *n* digits are added and the sum occupies *n +1* digits.
  - If *V* = 0 -> no overflow: *n*-bit results is correct.
  - If *V* = 1 -> overflow: The result contains *n* + *1* bits, and the *(n+1)*th bit is the actual sign.



Magnitude Comparator

- A *magnitude comparator* is a combinational circuit that compares two numbers, *A* and *B*, and then determines their relative magnitudes.

$A > B$

$A = B$

$A < B$

<u>Algorithm</u> Consider two numbers, A and B, with four digits each:

$$A = A_3A_2A_1A_0$$

$$B = B_3B_2B_1B_0$$

$x_i = 1$ if $A = B = 0$ or $A = B = 1$

$x_i = A_iB_i + A_i'B_i'$ for $i = 0, 1, 2, 3$ XNOR

For equality to exist, all x_i variables must be equal to 1:

$(A = B) = x_3x_2x_1x_0$ $\rightarrow$ AND operation

Magnitude Comparator (Contd.)

To determine if A is greater than or less than B, we inspect the relative magnitudes of significant digits.

If the two digits are equal, we compare the next lower significant pair of digits. The comparison continues until a pair of unequal digits is reached.

The sequential comparison can be expressed by:

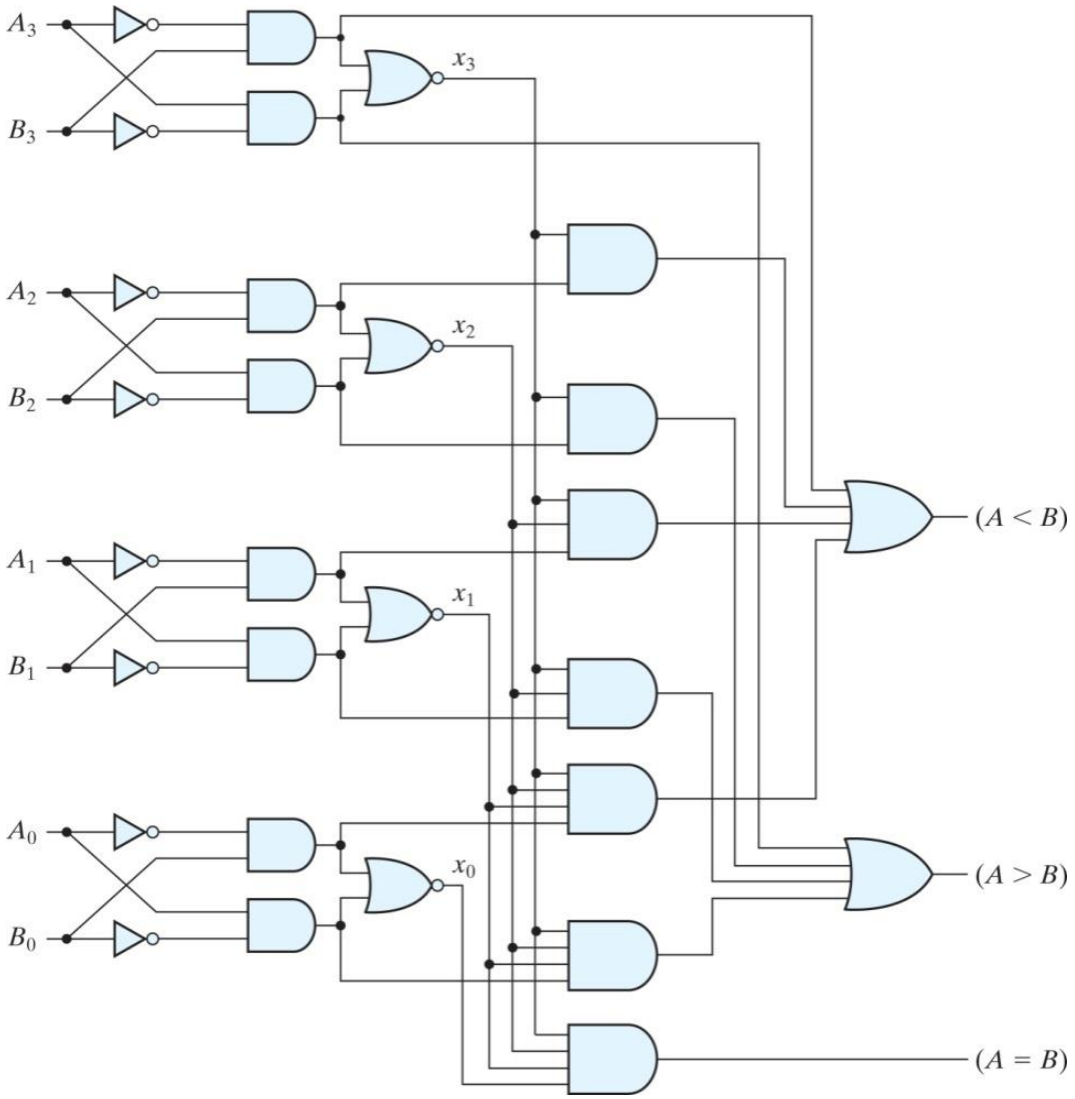
$$(A > B) = A_3 B_3' + x_3 A_2 B_2' + x_3 x_2 A_1 B_1' + x_3 x_2 x_1 A_0 B_0'$$

$$(A < B) = A_3' B_3 + x_3 A_2' B_2 + x_3 x_2 A_1' B_1 + x_3 x_2 x_1 A_0' B_0$$

Compare: $A = 1010$ and $B = 0101 \rightarrow (A > B) = 1$

$A = 0101$ and $B = 1010 \rightarrow (A < B) = 1$

Magnitude Comparator (Contd.)



Thank you!!!